Assignment 3: Creating a Virtual World (Hard)

- Due Sunday by 11:59pm
- Points 10
- Submitting a file upload

Videos:

- Prof. James' Tutorials:
  YouTube Playlist (https://www.youtube.com/watch?v=EBO9gk2feVY&list=PLbyTU_tFIkcOs7XVopOy5Oti-HGilZx0J)

(https://www.youtube.com/watch?v=EBO9gk2feVY&list=PLbyTU_tFIkcOs7XVopOy5Oti-HGilZx0J)

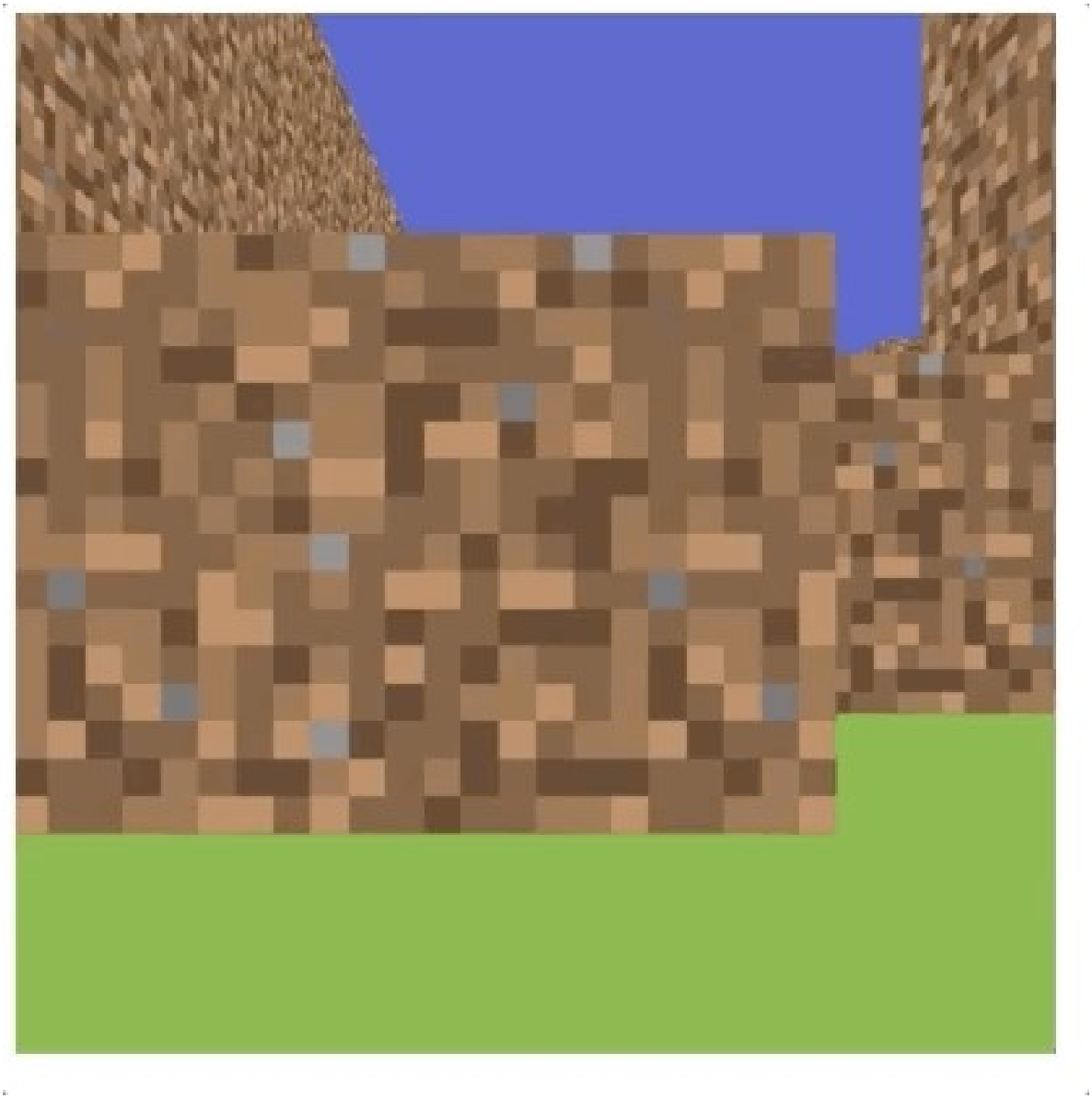
- Lab Section (Fall 2021): **Full video (Week 5) (https://media.ucsc.edu/V/Video?v=3830227&node=13160759&a=2061505220&autoplay=1)** || **Full video (Week 6) (https://media.ucsc.edu/V/Video?v=3858890&node=13245525&a=1361439544&autoplay=1)** || **YouTube Playlist (https://youtube.com/playlist?list=PLbyTU_tFIkcMK5l9lUnM0O3l3wmko8lPa)**
- Lab Section (Fall 2020) : **YouTube Playlist (https://youtube.com/playlist?list=PLbyTU_tFIkcP4KVBcJh71cdubBDQxUVkT)**

Objectives:

To create a virtual world using textured cubes and explore it using a perspective camera.

Introduction:

You will create a first-person exploration application. Your application should look like the following example:



A first-person camera (the player) should start in a given position of a 32x32x4 voxel-based 3D world (which is made out of textured cubes). The user should be able to navigate in this world using the keyboard and/or mouse to move and rotate the camera. Your application is required to have the following features:

- The world is fully created when the application starts.
  - Ground is created with a large plane (square), or the top of a flattened (scaled) cube.
  - Walls are created with cubes.
    - Walls can have different heights in units (1 cube, 2 cubes, 3 cubes and 4 cubes).
    - The faces of the cube should be textured to make it look like a wall.
  - The sky is created with a very large blue cube that is placed at the center of the world
  - The world layout should be specified using a hardcoded javascript 2D array.
    - Each element of the array represents the height of the wall (0, 1, 2, 3 or 4) that will be placed at that location.

- Camera can be moved using the keyboard.
  - W moves camera forward.
  - A moves camera to the left.
  - S moves camera backwards
  - D moves camera to the right.
  - Q turn camera left
  - E turn camera right

Additional requirements without helper videos:

- Camera can be rotated with the mouse.
- Add multiple textures to make your world more interesting.
- Simple Minecraft: add and delete blocks in front of you.
- Add your animal(s) to your world
- Change your ground plane from a flat plane to a terrain map OR get OBJ loader working
- Add some sort of simple story or game to your world

Instructions:

The following will walk you through each point of the basic rubric. Note that the written suggestions and video suggestions are not exactly the same. One may suggest to build a Class, while one just sticks a global variable. Or one may stick an 'if' statement in the shader while the other suggests an interpolation. Either is fine, or your own method is fine. You should do what you think makes cleaner better code.

0. Start a local web server.

Before we can load a texture file, we are going to need a web server. You can't just click the HTML file anymore because of security issues. The book gives a way to bypass security and just use the browser, but you shouldn't bypass security. Make sure you can view your webpage through this local server.

There is no rubric point for this since its not observable in your final output.

The helper videos (which are a couple years old) talk about setting up a Python HTTP server. You don't need to do this. We instead recommend the Live Preview extension for VS Code (which you should already have installed if you've followed our **Local Setup guide (https://canvas.ucsc.edu/courses/92112/pages/local-setup)** for the past few assignments).

- Texture Mapping (Matsuda Chapter 5)

This section will guide you to solve the following rubric points:

- Texture working on at least one object.

- Texture on some objects and color on some other objects. All working together.

1. Create a new attribute variable to store texture (UV) coordinates.

- Read Matsuda pages 137-145.
- Decide whether you want to use an extra buffer (textbook pages 137-140) or interleave a single buffer (textbook pages 141-145).
  - Either works, but make sure you can get this new attribute variable working.
  - You should be able to give each vertex a different UV coordinate.

There is no rubric point for this since its not observable in your final output*, but you need it working to get the texture working*.

2. Load Texture from the filesystem.

- Read Matsuda pages 146-178.
- Use the TexturedQuad example (functions *initTextures* and *loadTexture*) to load a texture file (e.g. sky.jpg) from your filesystem.
- Your texture must be square and size is a power of 2. e.g. 64x64 or 1024x1024. Resize it with an external program if you need to.

3. Pass the Texture to the fragment shader.

- Read Matsuda pages 179-181.
- Use the TexturedQuad example (fragment shader) to look up colors from texture.

4. Mix base color with texture color.

Your sky will be a gigantic cube with solid blue color surrounding the world. Your walls would be made of unit boxes that are textured. Thus, you need to use solid base colors in one object and textures in others.

- Create a new uniform variable (e.g. called *u_texColorWeight*) to linear interpolate between a given base color and the texture color.
  - For example, assuming the base color is stored in a uniform *baseColor* and the texture color in another variable *texColor.* You can linear between this two colors as follows:
    - t = *u_texColorWeight*
      gl_FragColor = (1 - t) * base*Color* + *t* * *texColor*
  - Note that you can use *u_texColorWeight=0* if you want only base color (for the sky box) *and u_baseColorWeight=1* if you want only texture colors (for the walls).

- Camera (Matsuda Chapter 7)

This section will guide you to solve the following rubric points:

- Implement camera movement.
- Key commands for rotation work. Use Q (rotate left) & E (rotate right).

- Perspective camera implemented.

5. Include both View and Projection matrices in your vertex shader.

- Read Matsuda pages 179-181.
- Use example "PerspectiveView_mvp" (vertex shader) to update your vertex shader to:
  - glPosition = u_ProjectionMatrix * u_ViewMatrix * u_ModelMatrix * a_position;
  - Note that the book has lots of examples that have some matrices, but not others. I believe this project is conceptually easier if you *pass all three as uniforms to the vertex shader and multiply them there*.

6. Create a Camera class to store and control both View and Projection matrices.

- Create a new file called camera.js and define a class called Camera that has the following attributes:
  - fov (field of view - float), initialize it to 60.
  - eye (Vector3), initialize it to (0,0,0).
  - at (Vector3), initialize it to (0,0,-1).
  - up (Vector3), initialize it to (0,1, 0).
  - viewMatrix (Matrix4), initialize it with viewMatrix.setLookAt(eye.elements[0], ... at.elements[0], ..., up.elements[0], ...).
  - projectionMatrix (Matrix4), initialize it with projectionMatrix.setPerspective(fov, canvas.width/ canvas.height, 0.1, 1000).
- In your camera class, create a function called "*moveForward"*:
  - Compute forward vector *f = at - eye*.
    - Create a new vector f: *let f = new Vector3();*
    - Set f to be equal to at: *f.set(at);*
    - Subtract eye from f: *f.sub(eye);*
  - Normalize f using *f.normalize();*
  - Scale f by a desired "speed" value: *f.mul(speed)*
  - Add forward vector to both eye and center: *eye += f; at += f;*
- In your camera class, create a function called "*moveBackwards"*:
  - Same idea as *moveForward,* but compute backward vector *b = eye - at* instead of forward.
- In your camera class, create a function called "move*Left"*:
  - Compute forward vector *f = at - eye.*
  - Compute side vector *s = up x f* (cross product between up and forward vectors).
  - Normalize s using *s.normalize();*
  - Scale s by a desired "speed" value: *s.mul(speed)*
  - Add side vector to both eye and center: *eye += s; at += s;*
- In your camera class, create a function called "move*Right"*:
  - Same idea as *moveLeft,* but compute the opposite side vector *s = f x up*.

- In your camera class, create a function called "pan*Left"*:
  - Compute the forward vector *f = at - eye*;
  - Rotate the vector f by alpha (decide a value) degrees around the up vector.
    - Create a rotation matrix: *rotationMatrix.setRotate(alpha, up.x, up.y, up.z)*.
    - Multiply this matrix by f to compute *f_prime = rotationMatrix.multiplyVector3(f);*
  - Update the "at"vector to be *at = eye + f_prime*;
- In your camera class, create a function called "panRight*"*:
  - Same idea as pan*Left,* but rotate u by -alpha degrees around the up vector.

7. Set View matrix using keyboard input.

- In your main function, instantiate a global camera object.
  - camera = new Camera()
- When user hits W, call camera.*moveForward()*
- When user hits S, call camera.*moveBackwards()*
- When user hits A, call camera.move*Left()*
- When user hits Q call camera.panLeft*()*
- *When user hits E call camera.panRight()*

- World Creation

Make sure you have your camera working already with keyboard control. If not, go do that section first. You can't debug your world if you cant actually see all of it.

This section will guide you to solve the following rubric points:

- Have a ground created with a flattened cube and sky from a big cube.
- World is implemented. There is some interesting world (or terrain) to walk around.

8. Add a ground to the world.

- Create a cube using the code you wrote in Assignment 2.
- Scale the cube on the y axis to make it a flat plane.
- Rotate this cube to put it in the x-z plane.

9. Add a sky box to the world.

- Create a cube using the code you wrote in Assignment 2.
- Place this cube in the center of the world and scale it by a very high number (e.g. 1000).

10. Add walls to the world.

Placing cubes manually in the world will genuinely suck if you need 100s of cubes to specify your world. Instead we will build it programatically.

- Define a small 2D array: *map = [[1 0 0 1] [1 1 0 1] [1 0 0 1] [1 1 1 1]]*.
- Write a double loop to create a wall wherever there is a 1.

```
walls = [];
for x = 1 ... 4:
   for z = 1 .. 4:
      if map[i][j] == 1 {
           let w = new Cube();
           w.translate(x, 0, z);
           walls.push(w);
      }
   }
}.
```

- Expand your map to 32x32.
- Instead of just making a 1 unit wall, make the wall N units high according to what you put in the 2d map array.
    - Include a third for loop to iterate on the height and translate the y coordinates of the walls.

You don't need to specify your map exactly like this if you want to be more fancy. Making just walls wont allow your world to have ceilings. You just need to end up with a world that is at least 32x32. You can't just write 100 hardcoded calls to drawCube() in your code. You should be able to 'edit the map'.

- Requirements without helper videos

Rotate camera with mouse.

- Most games have this control. You need to add an onMove() function and map this to the rotation you previously had on QE keys. Its a little tricky since with keys you can just turn 5degrees, but with mouse you have to keep track of the old mouse location and figure out an appropriate amount to turn based on the new mouse location.

Multiple textures

The book talks about multiple textures on pg 183. The easiest way to handle multiple textures is to just set up the samplers and texture units for all the textures and then just leave them (you dont need to pass all the textures with each drawArrays(), they will stay there). Just use a uniform variable to specify which texture to use and update this uniform before each call to drawCube(). Now different cubes can have different textures. You'll have to modify your map to say which kind of cube to draw as well. You may want to update your sky to have a different texture and ground as well. You can have up to 8 textures at a time loaded in webGL.

Simple Minecraft

Add and delete blocks in front of you. You do not need a raytracer, and you dont need full 3D pointing. You know your camera location and you know your map. Just find the map square right in front of you and add or delete a block from the stack of blocks that is there. Its super cool if you can save/load whatever world your create, but not required.

Add some sort of simple story or game to your world

e.g. Your animal takes care of a herd of baby animals, or pacman, or mini-minecraft and the mob wants to get you, or your animal puts on a puppet show as a story plays,. Use your creativity!

Performance

You may have efficiency issues, most people do on this assignment. If this is the case reduce world size until you can debug and interact. However to get this rubric point, you need to run 10fps with a full 32x32 world.

Wow!

This is intentionally vague. Its meant to leave you some space to do something you want to spend time on. You could make an impressive beautiful world. Past versions of this assignment talked about adding OBJ files or terrain maps to make it unique. But whatever you do, your goal is Wow!

FAQ:

- But I want to load my world from a file. Hard coding is stupid! - Yep. Hard coding is stupid. But its much easier. Load from a file if you want to do that. Then you can even load different 'game levels' from different files.
- How can I improve performance? - The primary issue is likely that you are rendering only one triangle at a time (since thats how the videos have showed it). That is, calling drawArrays() for each triangle. This is inefficient. It is possible to stack all the triangles you want to draw into a single vertexList and pass the whole thing to a single drawArrays() call. This will be much more efficient. The other big efficiency issue is allocating memory that is then garbage collected. Minimize the number of times new() is called in your code. If its in your rendering loop and happening every frame, then its probably too much.

Resources:

- (WebGL) Matsuda/Lea Ch5 (About texture)
- (WebGL) Matsuda/Lea Ch7 (About cameras and keyboard events (you dont need pg 276-289 about drawElements())
- (WebGL) Camera Movement Tutorial: **http://learnwebgl.brown37.net/07_cameras/camera_movement.html (http://learnwebgl.brown37.net/07_cameras/camera_movement.html)**
- (HTML/Javascript) How to handle keyboard events: **https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent (https://developer.mozilla.org/en-US/docs/Web/API/KeyboardEvent)**

What to Turn in:

(%24CANVAS_COURSE_REFERENCE%24/file_ref/gaa68d350951bf96b83681a987bb395f3/download)

1. **Canvas Submission**

Zip your entire project and submit it to Canvas under the appropriate assignment. Name your zip file "[FirstName]_[LastName]_Assignment_0.zip" (e.g. "Lucas_Ferreira_Assignment_0.zip").

2. **Live Hosted Submission**

You will upload your submission to GitHub Pages (or any other service of your choosing). If you use GitHub Pages, **click here (https://canvas.ucsc.edu/courses/92112/pages/submission-guide?module_item_id=1965025)** to learn how to set it up.

WHEN SUBMITTING YOUR PROJECT ON CANVAS, PLACE YOUR SITE LINK AS A COMMENT OF THE SUBMISSION.

Read the **Submission Guide (https://canvas.ucsc.edu/courses/92112/pages/submission-guide?module_item_id=1965025)** for further explanation on how to submit your assignment.

Assignment 3 Rubric (2)

Criteria	Ratings		Pts
Have a ground created with a flattened cube and sky from a big cube.	**0.5 pts Full Marks**	**0 pts No Marks**	0.5 pts
Texture working on at least one object.	**1 pts Full Marks**	**0 pts No Marks**	1 pts
Texture on some objects and color on some other objects. All working together.	**0.5 pts Full Marks**	**0 pts No Marks**	0.5 pts
Implement camera movement. W - move the camera forward; A - move the camera to the left; S - move the camera backwards; D - move the camera to the right;	**1 pts Full Marks**	**0 pts No Marks**	1 pts
Key commands for rotation work. Use Q (rotate left) & E (rotate right).	**1 pts Full Marks**	**0 pts No Marks**	1 pts
Perspective camera implemented.	**0.5 pts Full Marks**	**0 pts No Marks**	0.5 pts
World is implemented. There is some interesting world to walk around.	**1.5 pts Full Marks**	**0 pts No Marks**	1.5 pts

Criteria	Ratings		Pts
Rotate camera with the mouse Should work like most game controls.	**0.5 pts** **Full Marks**	**0 pts** **No Marks**	0.5 pts
Multiple textures There are at least two different textures in your world	**1 pts** **Full Marks**	**0 pts** **No Marks**	1 pts
Add/delete blocks	**0.5 pts** **Full Marks**	**0 pts** **No Marks**	0.5 pts
Add simple story or game to world	**1 pts** **Full Marks**	**0 pts** **No Marks**	1 pts
Performance 10fps without crazy lagging behavior	**0.5 pts** **Full Marks**	**0 pts** **No Marks**	0.5 pts
Wow! That extra something that impresses the grader! Beautiful world or OBJ Loader or Terrain or anything else that excites you to work on. Present it well and impress us!	**0.5 pts** **Full Marks**	**0 pts** **No Marks**	0.5 pts
			Total Points: 10